Bone Fracture Classification using Deep Learning

This project aims to automatically detect bone fractures from X-ray images using deep learning techniques.

A convolutional neural network (CNN) with transfer learning is used to classify X-ray images into two categories:

- **Fractured**
- **Not Fractured**

The system is designed for research and educational purposes and demonstrates how deep learning can assist in medical image analysis.

Environment Setup

The project is implemented using **PyTorch** and trained on GPU when available. The device is automatically selected to ensure optimal performance.

Key libraries used:

- PyTorch
- Torchvision
- OpenCV
- NumPy
- Matplotlib
- Scikit-learn

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, models

from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_curve,
    auc
)
```

```
# device = "cpu"
# # model = model.to(device)

device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)
```

```
Using device: cuda
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).
```

```
%cd /content/drive/MyDrive/DL_Project
```

```
/content/drive/MyDrive/DL_Project
```

Constants and Configuration

The following parameters are used throughout the project:

- Image size: 224 × 224
- Batch size: 32
- Learning rate: 1e-4
- Number of epochs: 20

These values were selected based on common practices in medical image classification and transfer learning.

```
IMG_SIZE = 224
DATA_DIR = "dataset"

TRAIN_DIR = "train"
VAL_DIR   = "val"
TEST_DIR  = "test"

BATCH_SIZE = 32
EPOCHS = 20
LR = 1e-4
```

Image Preprocessing (CLAHE)

X-ray images often suffer from low contrast, which can hide important details. To address this, **CLAHE (Contrast Limited Adaptive Histogram Equalization)** is applied to enhance bone structures and fracture patterns.

This step improves the visibility of edges and fine details while preventing over-amplification of noise.

```
def equalize(img):
    img = img.astype("uint8")
    clahe = cv2.createCLAHE(tileGridSize=(8,8))
    img = clahe.apply(img)
    img = img / 255.0
    return img
```

Data Augmentation

To reduce overfitting and improve generalization, data augmentation is applied **only to the training set**.

The following augmentations are used:

- Random rotation
- Random resized cropping
- Horizontal flipping
- Brightness and contrast jittering

Validation and test sets are kept unchanged to ensure fair evaluation.

```
train_transforms = transforms.Compose([
    transforms.ToPILImage(),
    transforms.RandomRotation(20),
    transforms.RandomResizedCrop(224, scale=(0.85, 1.0)),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor()
])

val_test_transforms = transforms.Compose([
    transforms.ToPILImage(),
    transforms.Resize((224,224)),
    transforms.ToTensor()
])
```

Dataset Preparation

A custom PyTorch `Dataset` class is implemented to:

- Load X-ray images from disk
- Convert images to grayscale
- Apply preprocessing and transformations
- Assign binary labels:
  - 1 → Fractured
  - 0 → Not Fractured

This approach provides full control over the data pipeline.

```python
class BoneDataset(Dataset):
    def __init__(self, root_dir, transform=None):
        self.samples = []
        self.transform = transform
        self.class_names = ["fractured", "not fractured"]

        for label, cls in enumerate(self.class_names):
            cls_dir = os.path.join(root_dir, cls)
            for img_name in os.listdir(cls_dir):
                self.samples.append(
                    (os.path.join(cls_dir, img_name), label)
                )

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        path, label = self.samples[idx]

        img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
        img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
        img = equalize(img)
        img = (img * 255).astype("uint8")

        if self.transform:
            img = self.transform(img)

        label = torch.tensor([label], dtype=torch.float32)
        return img, label
```

DataLoaders

PyTorch `DataLoader` objects are created for:

- Training
- Validation
- Testing

Data is shuffled during training to improve learning stability, while validation and test loaders preserve the original data order.

```python
train_ds = BoneDataset("dataset/train", train_transforms)
val_ds   = BoneDataset("dataset/val", val_test_transforms)
test_ds  = BoneDataset("dataset/test", val_test_transforms)

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE,
shuffle=True)
val_loader   = DataLoader(val_ds, batch_size=BATCH_SIZE,
shuffle=False)
```

```
test_loader  = DataLoader(test_ds, batch_size=BATCH_SIZE,
shuffle=False)
```

Model Architecture

A **ResNet-18** architecture pre-trained on ImageNet is used as the backbone network.

Key modifications:

- The first convolutional layer is adapted for grayscale input.
- The final fully connected layer is replaced with a single neuron for binary classification.
- Sigmoid activation is applied during inference.

Transfer learning allows the model to leverage rich feature representations learned from large-scale datasets.

```
class BoneResNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = models.resnet18(weights="IMAGENET1K_V1")

        # grayscale input
        self.model.conv1 = nn.Conv2d(
            1, 64, kernel_size=7, stride=2, padding=3, bias=False
        )

        self.model.fc = nn.Linear(self.model.fc.in_features, 1)

    def forward(self, x):
        return self.model(x)

model = BoneResNet().to(device)


criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LR)
```

Training Strategy

The model is trained using:

- Binary Cross-Entropy Loss with logits
- Adam optimizer
- Early stopping based on validation loss

Early stopping helps prevent overfitting by stopping training when validation performance no longer improves.

```python
train_losses, val_losses = [], []
train_accs, val_accs = [], []

def train_epoch(loader):
    model.train()
    total_loss, correct, total = 0, 0, 0

    for x, y in loader:
        x = x.to(device)
        y = y.to(device)

        optimizer.zero_grad()
        logits = model(x)
        loss = criterion(logits, y)
        loss.backward()
        optimizer.step()

        probs = torch.sigmoid(logits)
        preds = probs > 0.5

        total_loss += loss.item()
        correct += (preds == y).sum().item()
        total += y.size(0)

    return total_loss / len(loader), correct / total

def eval_epoch(loader):
    model.eval()
    total_loss, correct, total = 0, 0, 0

    with torch.no_grad():
        for x, y in loader:
            x = x.to(device)
            y = y.to(device)

            logits = model(x)
            loss = criterion(logits, y)

            probs = torch.sigmoid(logits)
            preds = probs > 0.5

            total_loss += loss.item()
            correct += (preds == y).sum().item()
            total += y.size(0)

    return total_loss / len(loader), correct / total

best_loss = float("inf")
patience = 5
counter = 0
```

```python
for epoch in range(EPOCHS):
    tr_loss, tr_acc = train_epoch(train_loader)
    vl_loss, vl_acc = eval_epoch(val_loader)

    train_losses.append(tr_loss)
    val_losses.append(vl_loss)
    train_accs.append(tr_acc)
    val_accs.append(vl_acc)

    print(f"Epoch {epoch+1}/{EPOCHS}")
    print(f"Train Loss: {tr_loss:.4f} | Train Acc: {tr_acc:.4f}")
    print(f"Val   Loss: {vl_loss:.4f} | Val   Acc: {vl_acc:.4f}")
    print("-"*40)

    if vl_loss < best_loss:
        best_loss = vl_loss
        counter = 0
        torch.save(model.state_dict(), "best_model.pth")
    else:
        counter += 1
        if counter == patience:
            print("Early stopping triggered")
            break
```

```
Epoch 1/20
Train Loss: 0.4053 | Train Acc: 0.7984
Val   Loss: 0.5340 | Val   Acc: 0.7399
----------------------------------------
Epoch 2/20
Train Loss: 0.1647 | Train Acc: 0.9354
Val   Loss: 0.1542 | Val   Acc: 0.9480
----------------------------------------
Epoch 3/20
Train Loss: 0.1033 | Train Acc: 0.9650
Val   Loss: 0.1250 | Val   Acc: 0.9191
----------------------------------------
Epoch 4/20
Train Loss: 0.0756 | Train Acc: 0.9759
Val   Loss: 0.0654 | Val   Acc: 0.9884
----------------------------------------
Epoch 5/20
Train Loss: 0.0692 | Train Acc: 0.9728
Val   Loss: 0.0727 | Val   Acc: 0.9653
----------------------------------------
Epoch 6/20
Train Loss: 0.0969 | Train Acc: 0.9805
Val   Loss: 0.0935 | Val   Acc: 0.9653
----------------------------------------
Epoch 7/20
Train Loss: 0.0645 | Train Acc: 0.9798
```

```
Val   Loss: 0.0559 | Val   Acc: 0.9827
----------------------------------------
Epoch 8/20
Train Loss: 0.0330 | Train Acc: 0.9899
Val   Loss: 0.0411 | Val   Acc: 0.9884
----------------------------------------
Epoch 9/20
Train Loss: 0.0310 | Train Acc: 0.9914
Val   Loss: 0.0219 | Val   Acc: 0.9884
----------------------------------------
Epoch 10/20
Train Loss: 0.0788 | Train Acc: 0.9821
Val   Loss: 0.0587 | Val   Acc: 0.9827
----------------------------------------
Epoch 11/20
Train Loss: 0.0489 | Train Acc: 0.9829
Val   Loss: 0.0829 | Val   Acc: 0.9711
----------------------------------------
Epoch 12/20
Train Loss: 0.0232 | Train Acc: 0.9946
Val   Loss: 0.0339 | Val   Acc: 0.9942
----------------------------------------
Epoch 13/20
Train Loss: 0.0183 | Train Acc: 0.9953
Val   Loss: 0.0299 | Val   Acc: 0.9942
----------------------------------------
Epoch 14/20
Train Loss: 0.0120 | Train Acc: 0.9969
Val   Loss: 0.0318 | Val   Acc: 0.9942
----------------------------------------
Early stopping triggered
```

```
torch.save(model.state_dict(), "bone_fracture_model.pth")
```

Training Curves

Training and validation accuracy and loss curves are plotted to analyze model behavior.

These curves help identify:

- Overfitting
- Underfitting
- Convergence stability

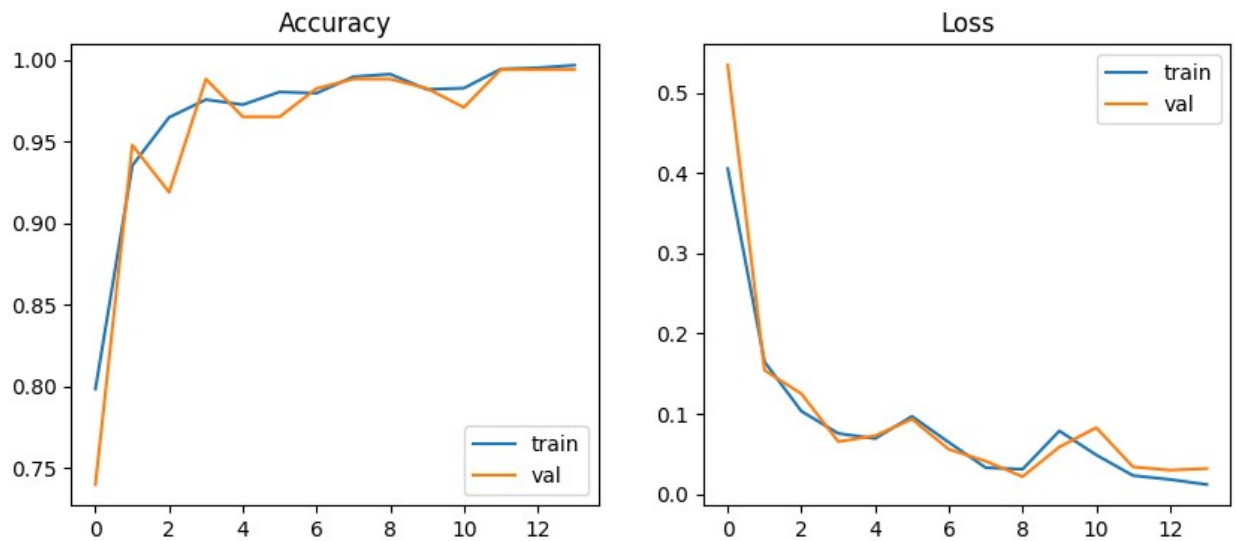
```
plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.plot(train_accs, label="train")
plt.plot(val_accs, label="val")
plt.title("Accuracy")
```

```
plt.legend()

plt.subplot(1,2,2)
plt.plot(train_losses, label="train")
plt.plot(val_losses, label="val")
plt.title("Loss")
plt.legend()

plt.show()
```



Model Evaluation

The trained model is evaluated on the test set using:

- Classification Report (Precision, Recall, F1-score)
- Confusion Matrix
- ROC Curve and AUC score

These metrics provide a comprehensive assessment of model performance, especially for medical classification tasks.

```
model.load_state_dict(torch.load("best_model.pth"))
model.eval()

y_true, y_pred, y_prob = [], [], []

with torch.no_grad():
    for x, y in test_loader:
        x = x.to(device)
        logits = model(x)
        probs = torch.sigmoid(logits)
```

```
        y_true.extend(y.numpy())
        y_prob.extend(probs.cpu().numpy())
        y_pred.extend((probs > 0.5).cpu().numpy())

print(classification_report(
    y_true,
    y_pred,
    target_names=["fractured", "not fractured"]
))
```

```
              precision    recall  f1-score   support

    fractured       0.96      0.98      0.97       263
not fractured       0.98      0.96      0.97       303

     accuracy                           0.97       566
    macro avg       0.97      0.97      0.97       566
 weighted avg       0.97      0.97      0.97       566
```

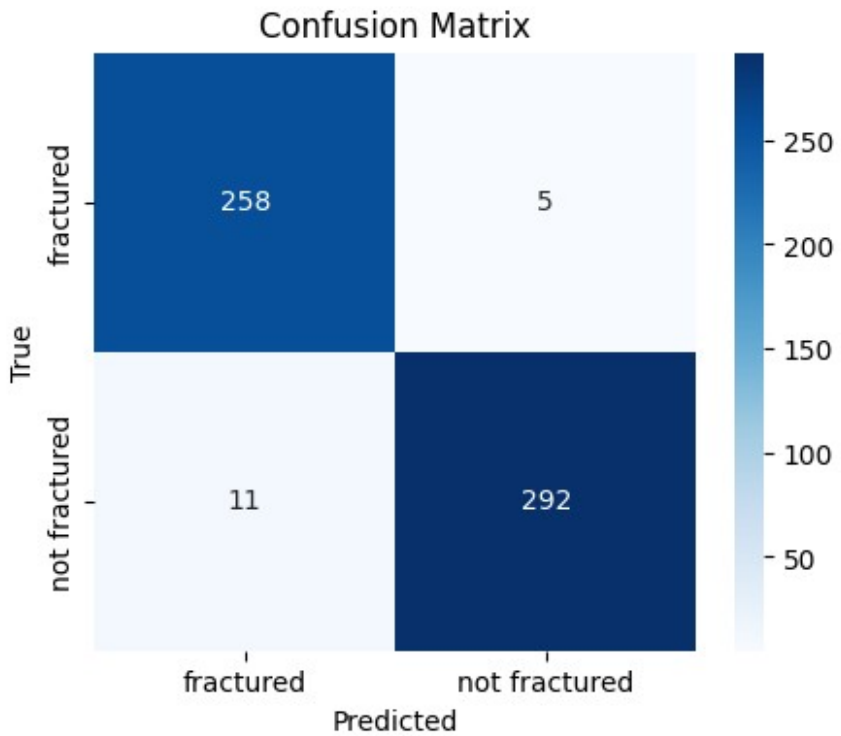
Confusion Matrix and ROC Curve

The confusion matrix visualizes the distribution of true positives, true negatives, false positives, and false negatives.

The ROC curve and AUC score measure the model's ability to distinguish between fractured and non-fractured cases across different thresholds.

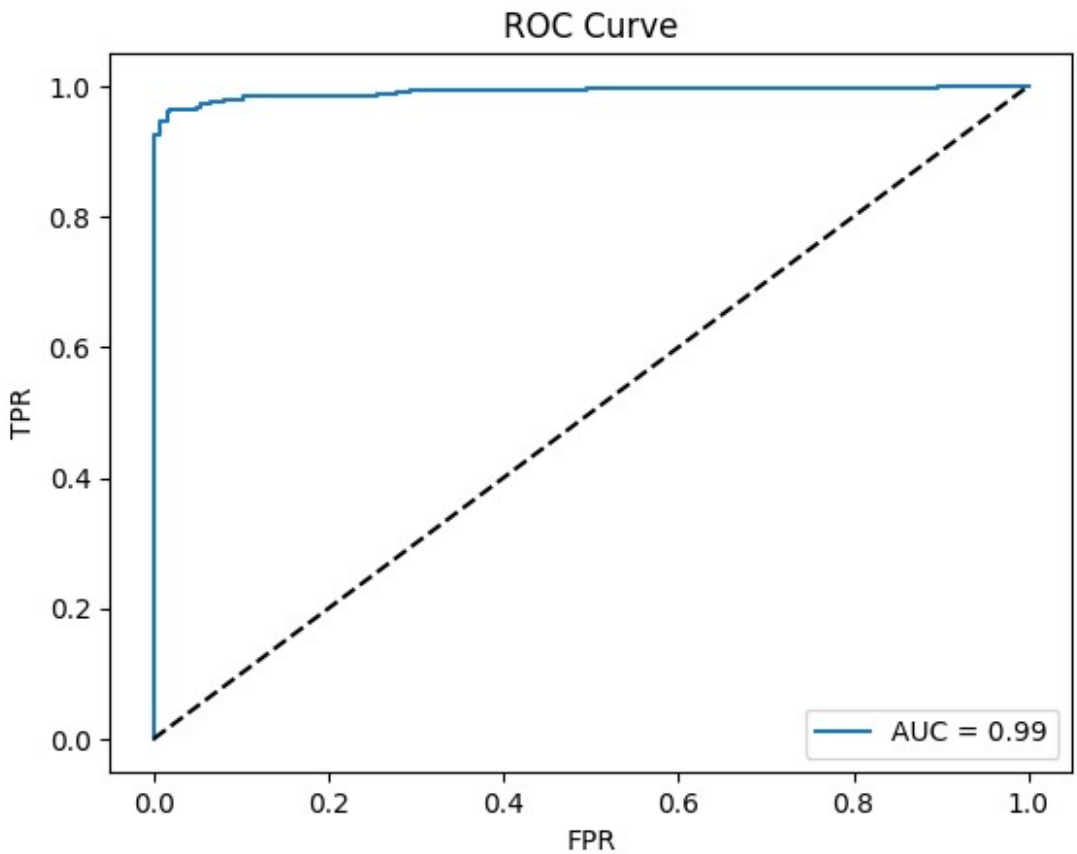
```
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(5,4))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["fractured", "not fractured"],
    yticklabels=["fractured", "not fractured"]
)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix")
plt.show()
```




```
fpr, tpr, _ = roc_curve(y_true, y_prob)
roc_auc = auc(fpr, tpr)

plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.2f}")
plt.plot([0,1], [0,1], "k--")
plt.xlabel("FPR")
plt.ylabel("TPR")
plt.title("ROC Curve")
plt.legend()
plt.show()
```



Single Image Prediction

A dedicated inference function is implemented to predict fracture presence for a single X-ray image.

The function:

- Applies the same preprocessing used during training
- Outputs the predicted class
- Reports a confidence score
- Uses a configurable decision threshold to balance sensitivity and specificity

```
def predict_single_image(path, threshold=0.5):
    model.eval()

    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
    img = equalize(img)

    img = np.expand_dims(img, axis=(0,1))
    img = torch.tensor(img, dtype=torch.float32).to(device)

    with torch.no_grad():
```

```
        logit = model(img)
        prob = torch.sigmoid(logit).item()

    if prob >= threshold:
        pred_class = "fractured"
        confidence = prob
    else:
        pred_class = "not fractured"
        confidence = 1 - prob

    return pred_class, confidence

pred_class, conf = predict_single_image(
    "dataset/test/fractured/92-rotated3-rotated3-rotated3.jpg"
)

print("Predicted Class:", pred_class)
print("Confidence:", conf)
```

```
Predicted Class: not fractured
Confidence: 0.9918937692418694
```